GUION DE LA PRESENTACIÓN

PLATAFORMA DE TENANTS «PRESENCIA CLOUD»

Demostración funcional y explicación del diseño, dividida por integrante

Proyecto final del módulo de DevOps e infraestructura como código

Elaborado por: Grupo GPT-6 AURA

Fecha: 12 de noviembre de 2026

ÍNDICE

1. ORGANIZACIÓN DEL VIDEO **3**

2. INTEGRANTE 1 · PLATAFORMA **5**

3. INTEGRANTE 2 · TERRAFORM **11**

4. INTEGRANTE 3 · SEGUNDO MOTOR Y CICLO DE VIDA **15**

5. INTEGRANTE 4 · PACKER, CLOUD-INIT Y ANSIBLE **20**

6. INTEGRANTE 5 · GRAFANA **27**

7. INTEGRANTE 6 · SEGURIDAD Y DOCUMENTACIÓN **33**

1. ORGANIZACIÓN DEL VIDEO

El docente pidió un video de no más de 15 minutos en el que cada integrante explique la parte que construyó, mostrando que funciona. Este guion divide la presentación en seis partes de hasta dos minutos, más una apertura y un cierre breves. Cada parte indica qué requisitos cubre, qué mostrar en pantalla, los comandos, las capturas y fragmentos de código de apoyo, cómo funciona por detrás y el monólogo sugerido.

Tabla 1. Orden y tiempos

Parte	Integrante y tema	Requisitos que demuestra	Tiempo
Apertura	Integrante 1	Qué es la plataforma y su arquitectura	0:30
2	Integrante 1 · Plataforma	RF-01 a RF-03, RF-07, RF-11, RF-14, CU-01, CU-03, CU-04	2:00
3	Integrante 2 · Terraform	RF-08, RF-09, RF-13; Terraform como motor principal	2:00
4	Integrante 3 · Pulumi y ciclo de vida	RF-08, RF-12, RF-16 a RF-18; CU-05 a CU-07	2:00
5	Integrante 4 · Packer, cloud-init y Ansible	RF-10; Packer, cloud-init y Ansible	2:00
6	Integrante 5 · Grafana	RF-25 a RF-29; CU-08, CU-09	2:00
7	Integrante 6 · Seguridad y documentación	Requisitos no funcionales, CU-14, entregables	2:00
Cierre	Integrante 6	Resumen y costos	0:30

1.1. Preparación antes de grabar

- Plataforma en AWS encendida: consola en `http://3.235.12.162` (usuario admin, contraseña admin1234!).
- Tenants ya creados y activos: **upsa** (Básico, Terraform) y **unifranz** (Estándar, Pulumi).
- Grafana abierto en `http://3.235.12.162/grafana/d/salud-tenants`.
- El repositorio abierto en el editor, para mostrar los archivos que se citan.
- Consola de AWS abierta en EC2, para entrar a los servidores con Connect → Session Manager.
- Crear un tenant demora entre 4 y 6 minutos: se inicia en la parte 2 y se pausa la grabación mientras termina, como sugirió el docente.

1.2. Arquitectura que se presenta

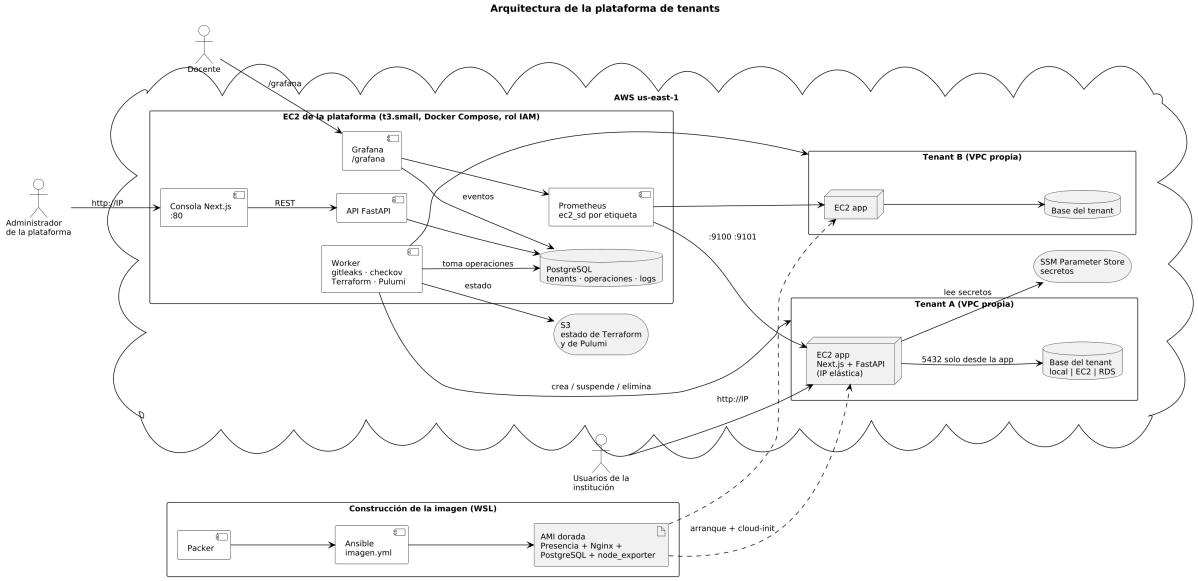


Figura 1. Arquitectura de la plataforma y de los tenants.

2. INTEGRANTE 1 · PLATAFORMA

Cubre: catálogo de planes (RF-01 a RF-03), creación del tenant (RF-07), progreso y errores comprensibles (RF-11), consulta por nombre, estado, plan y entorno (RF-14) y la ejecución en segundo plano. Es la parte que une a todas las demás.

Qué mostrar en pantalla

1. Abrir `http://3.235.12.162`: la página de inicio con los tres planes, todos a la misma altura, con precio, recursos e inclusiones.
2. Ingresar a la consola y mostrar el Panel: tenants activos, costo estimado por día y actividad.
3. Abrir Nuevo tenant, elegir el plan Básico y el motor Terraform, completar la institución (por ejemplo «Universidad Privada Domingo Savio», identificador upds) y presionar Crear tenant.
4. Mostrar el log en vivo de la operación (escaneo, Terraform) y pausar la grabación hasta que termine.
5. Mostrar Instituciones con los filtros y editar el nombre de una institución.
6. En el editor, mostrar planes.yaml y la función que toma operaciones de la cola.

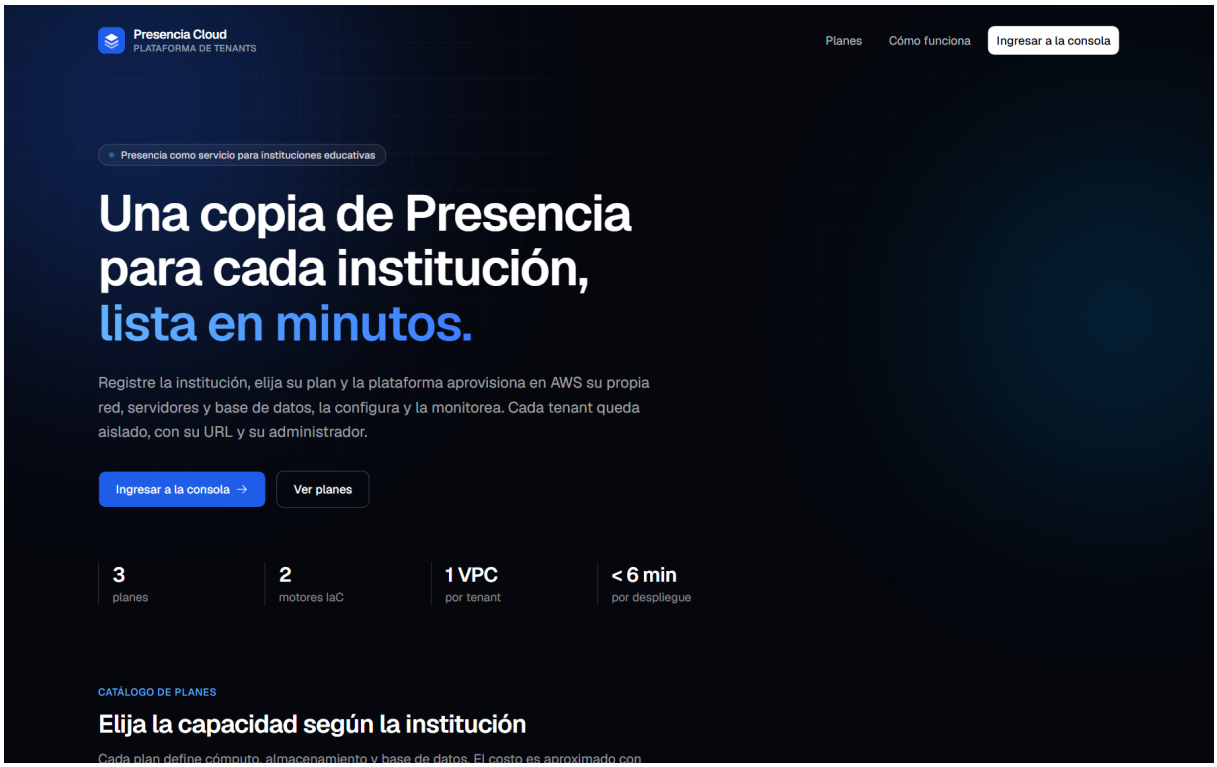


Figura 2. Página de inicio con el catálogo de planes.

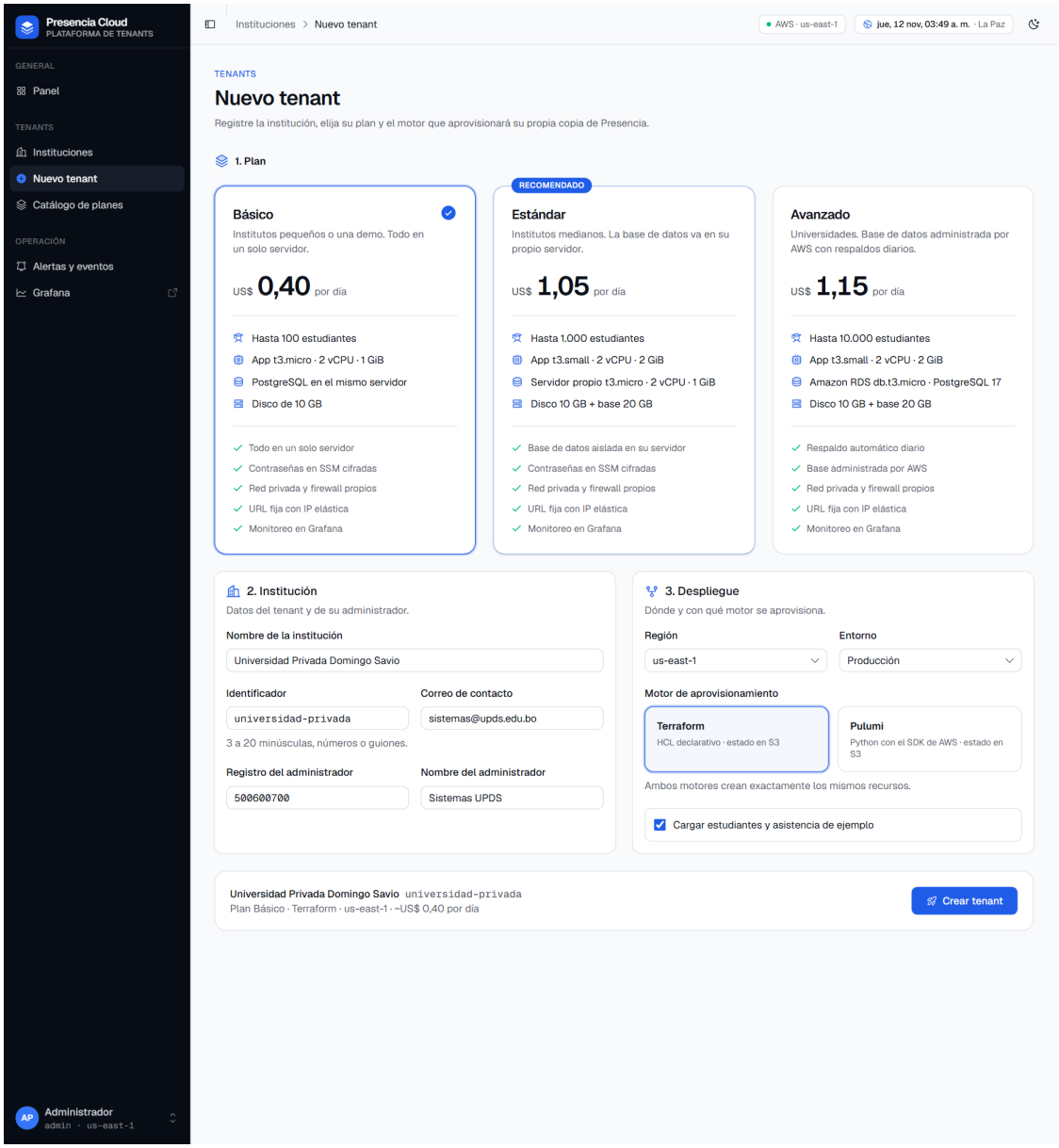


Figura 3. Formulario de un nuevo tenant: plan, institución y despliegue.

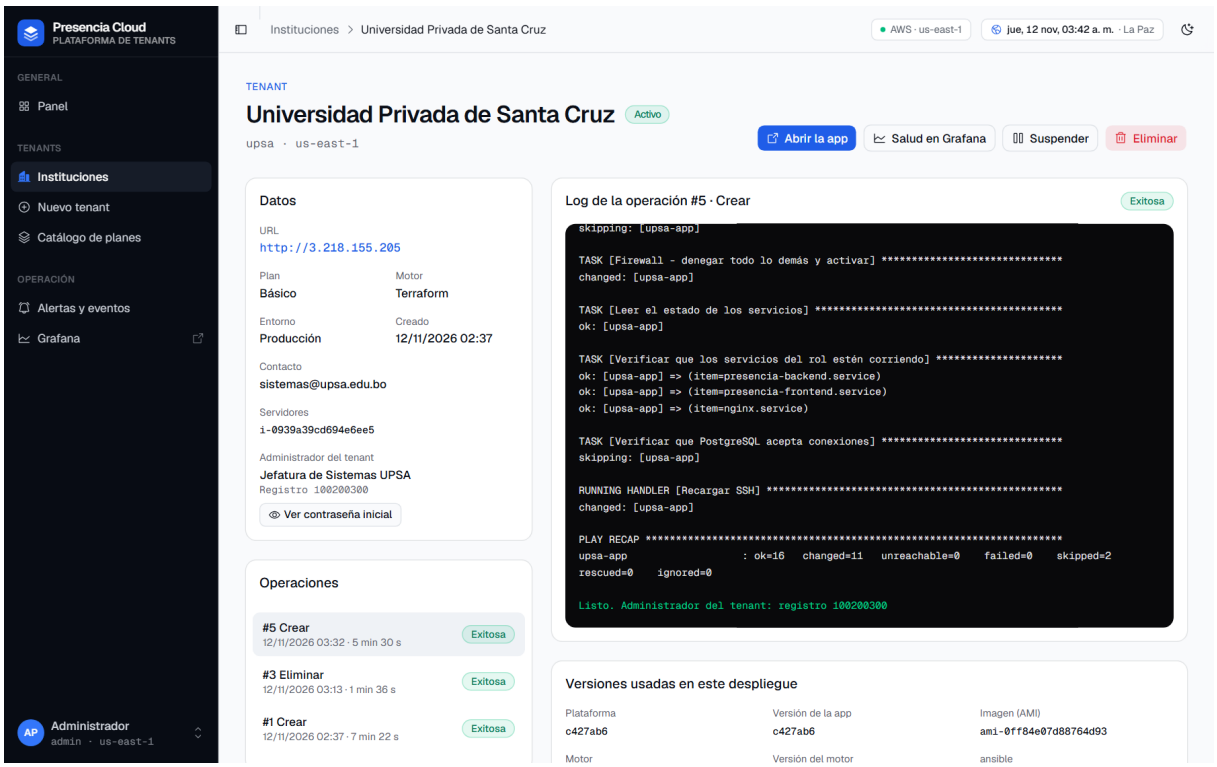


Figura 4. Detalle de un tenant con su log en vivo y las versiones registradas.

Cómo se construyó

Tabla 2. Construcción de la plataforma

Parte	Cómo está hecha	Qué se reutilizó de Presencia
Backend	FastAPI y SQLAlchemy sobre PostgreSQL: tenants, operaciones y su log. Endpoints para planes, tenants, ciclo de vida, log, eventos y alertas.	La misma arquitectura y estilo que el backend de Presencia (FastAPI, Pydantic, JWT).
Worker	Proceso aparte que toma operaciones pendientes con FOR UPDATE SKIP LOCKED y las ejecuta en hilos: dos tenants pueden crearse a la vez, pero nunca dos operaciones del mismo tenant.	—
Frontend	Next.js con landing pública, login y consola con menú lateral; consulta el log cada 2 segundos.	Componentes shadcn/ui, modal propio, KPI, encabezado, tema claro/oscuro y el proxy BFF con cookie httpOnly, copiados de Presencia.
Catálogo	plataforma/planes.yaml: un único archivo que leen la API, la consola, Terraform y Pulumi.	—
Despliegue	Docker Compose: backend, worker, frontend, base, Prometheus y Grafana.	—

```yaml
basico:
  nombre: Básico
  descripcion: Institutos pequeños o una demo. Todo en un solo servidor.
  costo_dia_usd: 0.40
  estudiantes: 100
  app:
    tipo: t3.micro
    vcpu: 2
    memoria_gib: 1
    disco_gb: 10
  bd:
    modo: local
    almacenamiento_gb: 10
    descripcion: PostgreSQL en el mismo servidor de la app
```

Figura 5. plataforma/planes.yaml, líneas 1 a 15. Un plan del catálogo: lo que ve el usuario es lo que se aprovisiona.

```python
def tomar_operacion() -> int | None:
    otra = aliased(Operacion)
    with SessionLocal() as db:
        op = db.scalar(
            select(Operacion)
            .where(Operacion.estado == EstadoOperacion.pendiente)
            .where(
                ~exists().where(
                    otra.tenant_id == Operacion.tenant_id,
                    otra.estado == EstadoOperacion.ejecutando,
                )
            )
            .order_by(Operacion.id)
            .limit(1)
            .with_for_update(skip_locked=True, of=Operacion)
        )
        if op is None:
            return None
        op.estado = EstadoOperacion.ejecutando
        op.iniciada_en = datetime.now(UTC)
        db.commit()
        return op.id
```

Figura 6. plataforma/backend/plataforma/worker.py, líneas 194 a 215. El worker toma la próxima operación pendiente sin bloquear a otros hilos.

```python
@app.post("/tenants", response_model=TenantSalida, status_code=201, tags=["Tenants"])
def crear(datos: TenantNuevo, db: Db, _: Admin) -> TenantSalida:
    if datos.region not in settings.regiones:
        raise HTTPException(status.HTTP_422_UNPROCESSABLE_ENTITY, "Región no habilitada")
    existente = db.get(Tenant, datos.id)
    if existente and existente.estado != EstadoTenant.eliminado:
        raise HTTPException(status.HTTP_409_CONFLICT, "Ya existe un tenant con ese identificador")
    if existente:
        for campo, valor in datos.model_dump().items():
            setattr(existente, campo, valor)
        existente.estado, existente.salidas, existente.url = EstadoTenant.creando, {}, None
        tenant = existente
    else:
        tenant = Tenant(**datos.model_dump(), estado=EstadoTenant.creando)
        db.add(tenant)
    db.flush()
    encolar(db, tenant, TipoOperacion.crear)
    return con_ultima(db, tenant)
```

Figura 7. plataforma/backend/plataforma/main.py, líneas 165 a 182. Crear un tenant solo registra la operación: el trabajo pesado lo hace el worker en segundo plano.

Cómo funciona por detrás

La consola nunca habla directo con la API: pasa por el proxy de Next.js, que agrega el token guardado en una cookie httpOnly. Al crear un tenant, la API valida los datos, guarda el tenant como «creando» y encola una operación. El worker la toma, va guardando cada línea de salida en la base y la consola las muestra cada 2 segundos. Los errores de AWS, Terraform o Pulumi se traducen a mensajes en español; el detalle técnico sigue en el log.

Monólogo

Lectura de aproximadamente 1,8 minutos (251 palabras a un ritmo de presentación de unas 140 palabras por minuto).

Yo me encargué de la plataforma, que es la pieza que une todo. Esta es la página de inicio: muestra el catálogo con los tres planes, Básico, Estándar y Avanzado, con sus recursos y su costo por día. Ese catálogo vive en un solo archivo, planes.yaml, que leen la consola, la API, Terraform y Pulumi; así lo que ofrecemos es exactamente lo que se aprovisiona.

Entro a la consola. El panel resume los tenants activos, los suspendidos y el costo estimado. Ahora creo un tenant: escribo el nombre de la institución, el identificador se sugiere solo, elijo el plan Básico y el motor Terraform, y presiono Crear tenant.

La creación no bloquea la pantalla: la API solo registra la operación en una cola en PostgreSQL. Un proceso aparte, el worker, la toma con FOR UPDATE SKIP LOCKED, que permite crear varios tenants a la vez sin que dos procesos tomen la misma tarea. Cada línea que imprimen las herramientas se guarda en la base, y la consola la muestra casi en tiempo real. Si algo falla, el error se traduce a un mensaje entendible.

El backend está hecho con FastAPI y el frontend con Next.js, reutilizando los mismos componentes y el mismo proxy con cookie segura que ya teníamos en Presencia. En la lista de instituciones puedo buscar por nombre y filtrar por estado, plan y entorno.

Todo corre con Docker Compose en una instancia EC2 de AWS. Mientras se crea este tenant, mi compañero explica cómo Terraform levanta su infraestructura.

3. INTEGRANTE 2 · TERRAFORM

Cubre: Terraform como motor principal (sección 4 de los requisitos), crear los recursos de frontend, backend y base de datos (RF-09) y el registro de versiones de cada despliegue (RF-13).

Qué mostrar en pantalla

1. En el log del tenant que se está creando, mostrar el escaneo y luego «Plan: 24 to add» y «Apply complete».
2. En el editor, abrir infraestructure/tenant/terraform/main.tf: el catálogo leído con yamldecode y los recursos que dependen del plan.
3. Mostrar versions.tf: el estado se guarda en S3, una clave por tenant.
4. Mostrar en la consola de AWS (VPC y EC2) los recursos con la etiqueta Tenant.
5. Mostrar en la pantalla del tenant el bloque «Versiones usadas en este despliegue».

```
$ terraform -chdir=/app/repo/infraestructure/tenant/terraform init -no-color -reconfigure -lockfile=readonly -backend-
$ terraform -chdir=/app/repo/infraestructure/tenant/terraform apply -auto-approve -no-color -var-file=/datos/terraform
Plan: 24 to add, 0 to change, 0 to destroy.
aws_key_pair.plataforma: Creating...
aws_ssm_parameter.secretos["bd_password"]: Creating...
aws_ssm_parameter.secretos["admin_password"]: Creating...
aws_ssm_parameter.secretos["jwt_secret"]: Creating...
aws_ssm_parameter.secretos["admin_password"]: Creation complete after 1s [id=/presencia/tenants/upsa/admin_password]
aws_key_pair.plataforma: Creation complete after 1s [id=presencia-upsa-plataforma]
aws_ssm_parameter.secretos["bd_password"]: Creation complete after 1s [id=/presencia/tenants/upsa/bd_password]
aws_ssm_parameter.secretos["jwt_secret"]: Creation complete after 1s [id=/presencia/tenants/upsa/jwt_secret]
module.app.aws_instance.this: Creation complete after 13s [id=i-0939a39cd694e6ee5]
aws_eip.app: Creation complete after 3s [id=eipalloc-0248e8f3f79f63432]
Apply complete! Resources: 24 added, 0 changed, 0 destroyed.
$ terraform -chdir=/app/repo/infraestructure/tenant/terraform output -json -no-color

Outputs:

admin_registro = "100200300"
ami = "ami-0ff84e07d88764d93"
instancias = [
  "i-0939a39cd694e6ee5",
]
ip_publica = "3.218.155.205"
rds = ""
servidores = {
  "app" = "3.218.155.205"
}
url = "http://3.218.155.205"
$ terraform -chdir=/app/repo/infraestructure/tenant/terraform output -json -no-color
```

Figura 8. Salida real de Terraform al crear el tenant upsa (plan Básico).

```
locals {
  catalogo = yamldecode(file("${path.module}/../../../plataforma/planes.yaml"))
  plan     = local.catalogo[var.plan]
  bd_modo  = local.plan.bd.modo
  prefijo  = "presencia-${var.tenant}"

  vpc_cidr     = "10.0.0.0/16"
  subred_app   = "10.0.1.0/24"
  subredes_rds = ["10.0.11.0/24", "10.0.12.0/24"]
  ip_app       = "10.0.1.10"
  ip_bd        = "10.0.1.30"

  ami_id = var.ami_id != "" ? var.ami_id : data.aws_ami.presencia.id

  bd_host = {
```

Figura 9. infraestructure/tenant/terraform/main.tf, líneas 1 a 15. El plan elegido sale del catálogo; según su base de datos se crean distintos recursos.

```
module "app" {
  source = "../../modules/ec2"

  name                 = "${local.prefijo}-app"
  role                 = "app"
  ami_id               = local.ami_id
  instance_type        = local.plan.app.tipo
  subnet_id            = module.network.public_subnet_id
  private_ip           = local.ip_app
  security_group_ids   = [aws_security_group.app.id]
  key_name             = aws_key_pair.plataforma.key_name
  volume_size          = local.plan.app.disco_gb
  associate_public_ip  = true
  iam_instance_profile = aws_iam_instance_profile.servidor.name
  user_data = templatefile("${path.module}/../cloud-init/servidor.yaml.tftpl", merge(local.cloud_init, {
    rol     = "app"
    bd_host = local.bd_host
  }))

  depends_on = [aws_ssm_parameter.secretos, aws_iam_role_policy.leer_secretos]
```

Figura 10. infraestructure/tenant/terraform/main.tf, líneas 314 a 333. El servidor de la app reutiliza el módulo ec2 del proyecto y recibe la plantilla de cloud-init.

```
  backend "s3" {
    bucket       = "presencia-tfstate-aliaga23"
    region       = "us-east-1"
    encrypt      = true
    use_lockfile = true
  }
```

Figura 11. infraestructure/tenant/terraform/versions.tf, líneas 16 a 21. El estado vive en S3 con bloqueo nativo.

```
103     def _preparar(self) -> Path:
104         archivo = self.datos / "tenant.tfvars.json"
105         archivo.write_text(json.dumps(self.variables, indent=2), encoding="utf-8")
106         self._tf(
107             "init",
108             "-no-color",
109             "-reconfigure",
110             "-lockfile=readonly",
111             f"-backend-config=key={self.clave_estado}",
112         )
113         return archivo
114 
115     def aplicar(self) -> dict:
116         archivo = self._preparar()
117         self._tf("apply", "-auto-approve", "-no-color", f"-var-file={archivo}")
118         salidas = json.loads(self._tf("output", "-json", "-no-color", mostrar=False))
119         return {clave: valor["value"] for clave, valor in salidas.items()}
```

Figura 12. plataforma/backend/plataforma/motores.py, líneas 103 a 119. Cómo el worker ejecuta Terraform: init con la clave del tenant, apply y lectura de salidas.

Tabla 3. Recursos que crea Terraform según el plan

Plan	Recursos principales
Básico	VPC, subred, grupos de seguridad, 1 EC2 con PostgreSQL local, IP elástica, rol IAM y secretos (24 recursos).
Estándar	Lo anterior más una EC2 de base de datos y su grupo de seguridad.
Avanzado	VPC con dos subredes privadas, grupo de subredes y una instancia Amazon RDS (30 recursos).

Tabla 4. Comandos

Para qué	Comando
Ver el estado de cada tenant en S3	`aws s3 ls s3://presencia-tfstate-aliaga23/tenants/`
Validar el módulo del tenant	`terraform -chdir=infraestructure/tenant/terraform validate`
Desplegar la plataforma	`terraform -chdir=infraestructure/environments/plataforma apply`

Cómo funciona por detrás

El worker escribe las variables del tenant en un archivo JSON, ejecuta terraform init indicando la clave de estado tenants/<id>/terraform.tfstate y luego apply. Como el estado registra lo que ya existe, repetir un apply nunca duplica recursos. Las salidas (URL, IP de cada servidor, instancias) se guardan en el tenant y las versiones usadas quedan en la operación.

Monólogo

Lectura de aproximadamente 1,8 minutos (248 palabras a un ritmo de presentación de unas 140 palabras por minuto).

Yo trabajé con Terraform, que es nuestro motor principal. Mientras se crea el tenant, en el log se ve cómo Terraform calcula el plan: para un plan Básico son 24 recursos. Primero la red propia del tenant, con su VPC, subred y tabla de rutas; después los grupos de seguridad, el rol IAM, los secretos en SSM Parameter Store, el servidor y su IP elástica.

El código está en infraestructure/tenant/terraform. Lo primero que hace es leer el catálogo con yamldecode, el mismo planes.yaml que usa la plataforma, así el tamaño de la máquina y el tipo de base salen del plan elegido. Con count y for_each decido qué crear: en el Básico, PostgreSQL va en el mismo servidor; en el Estándar se agrega un servidor de base de datos; y en el Avanzado se crea una base Amazon RDS con respaldo diario.

No empezamos de cero: reutilizamos los módulos de red y de EC2 que ya teníamos de los prácticos, con IMDSv2 obligatorio y discos cifrados.

El estado se guarda en un bucket S3, con una clave por tenant y bloqueo nativo. Eso permite tener muchos tenants sin que se mezclen, y que una operación se pueda repetir sin duplicar nada, porque Terraform compara contra el estado guardado.

Al terminar, la plataforma guarda las salidas, como la URL y las IP, y registra las versiones de la aplicación, la imagen, Terraform y la huella de las plantillas. Con el mismo Terraform también desplegamos la propia plataforma en AWS.

4. INTEGRANTE 3 · SEGUNDO MOTOR Y CICLO DE VIDA

Cubre: elegir el motor (RF-08) con Pulumi como segunda implementación, suspender y reanudar conservando los datos (RF-16, RF-17), eliminar mostrando lo que se borra (RF-18) y reintentar sin duplicar recursos (RF-12).

Qué mostrar en pantalla

1. Abrir el tenant unifranz (Estándar, creado con Pulumi) y mostrar su log de Pulumi.
2. Mostrar en el editor infraestructure/tenant/pulumi/__main__.py junto al main.tf de Terraform.
3. Presionar Suspender: el estado pasa a Suspendido y la app deja de responder.
4. Presionar Reanudar: vuelve en la misma URL con sus datos.
5. Presionar Eliminar sobre el tenant de prueba: mostrar la lista de recursos y la confirmación.
6. Mostrar la captura de una operación fallida y su reintento.

```
$ pulumi stack ls --json --cwd /app/repo/infraestructure/tenant/pulumi
$ pulumi stack init --cwd /app/repo/infraestructure/tenant/pulumi --stack tenant-unifranz --non-interactive --color ne
$ pulumi up --yes --skip-preview --config-file /datos/pulumi/unifranz/Pulumi.tenant-unifranz.yaml --cwd /app/repo/infr
Updating (tenant-unifranz):
 +  pulumi:providers:aws aws created (1s)
 +  aws:ec2:KeyPair plataforma created (4s)
 +  aws:iam:Role servidor-role created (4s)
 +  aws:ssm:Parameter admin_password created (4s)
 +  aws:ec2:Vpc vpc created (5s)
 +  random:index:RandomPassword jwt created (1s)
 +  random:index:RandomPassword bd created (1s)
 +  aws:iam:RolePolicyAttachment ssm-core created (2s)
 +  aws:iam:RolePolicy leer-secretos created (2s)
 +  aws:ssm:Parameter jwt_secret created (0.65s)
 +  aws:ec2:InternetGateway igw created (3s)
 +  aws:ec2:Subnet public-subnet created (1s)
 +  aws:ssm:Parameter bd_password created (0.84s)
 +  aws:ec2:SecurityGroup app-sg created (2s)
 +  aws:ec2:SecurityGroup bd-sg created (4s)
 +  aws:ec2:RouteTable public-rt created (1s)
 +  aws:vpc:SecurityGroupEgressRule app-internet-http created (2s)
 +  aws:vpc:SecurityGroupIngressRule app-http created (2s)
 +  aws:vpc:SecurityGroupIngressRule app-ssh-3.235.12.162/32 created (1s)
 +  aws:vpc:SecurityGroupIngressRule app-metricas-3.235.12.162/32 created (2s)
 +  aws:vpc:SecurityGroupEgressRule app-internet-ntp created (1s)
 +  aws:iam:InstanceProfile servidor-profile created (8s)
```

Figura 13. Salida real de Pulumi al crear el tenant unifranz (plan Estándar).

```python
def servidor(rol_servidor: str, tipo: str, ip: str, disco: int, sgs, user_data, depende) -> aws.ec2.Instance:
    """Igual que modules/ec2: IMDSv2, disco gp3 cifrado y creditos T3 estandar."""
    return aws.ec2.Instance(
        rol_servidor,
        ami=ami_id,
        instance_type=tipo,
        subnet_id=subred.id,
        private_ip=ip,
        associate_public_ip_address=True,
        vpc_security_group_ids=sgs,
        iam_instance_profile=perfil.name,
        key_name=clave_plataforma.key_name,
        user_data=user_data,
        credit_specification=aws.ec2.InstanceCreditSpecificationArgs(cpu_credits="standard"),
        metadata_options=aws.ec2.InstanceMetadataOptionsArgs(http_endpoint="enabled", http_tokens="required"),
        root_block_device=aws.ec2.InstanceRootBlockDeviceArgs(
            volume_type="gp3", volume_size=disco, encrypted=True, delete_on_termination=True
        ),
        tags={"Name": f"{prefijo}-{rol_servidor}", "Role": rol_servidor},
        opts=pulumi.ResourceOptions(provider=proveedor, depends_on=depende, ignore_changes=["ami"]),
```

Figura 14. infraestructure/tenant/pulumi/__main__.py, líneas 317 a 336. Pulumi en Python crea el mismo servidor que el módulo ec2 de Terraform.

```python
def suspender(region: str, salidas: dict, log: Log) -> None:
    ec2 = boto3.client("ec2", region_name=region)
    instancias = salidas.get("instancias") or []
    log(f"Apagando servidores: {', '.join(instancias)}")
    ec2.stop_instances(InstanceIds=instancias)
    ec2.get_waiter("instance_stopped").wait(InstanceIds=instancias)
    log("Servidores apagados. Los discos y los datos se conservan.")
    if salidas.get("rds"):
        rds = boto3.client("rds", region_name=region)
        log(f"Deteniendo la base RDS {salidas['rds']}")
        rds.stop_db_instance(DBInstanceIdentifier=salidas["rds"])
        _esperar_rds(rds, salidas["rds"], "stopped", log)
```

Figura 15. plataforma/backend/plataforma/nube.py, líneas 52 a 63. Suspender apaga las instancias y, en el plan Avanzado, también la base RDS.

```python
@app.post("/tenants/{tenant_id}/reintentar", response_model=OperacionSalida, tags=["Ciclo de vida"])
def reintentar(tenant_id: str, db: Db, _: Admin) -> Operacion:
    tenant = buscar(db, tenant_id)
    libre(tenant, db)
    ultima = db.scalar(
        select(Operacion).where(Operacion.tenant_id == tenant.id).order_by(Operacion.id.desc())
    )
    if ultima is None or ultima.estado != EstadoOperacion.fallida:
        raise HTTPException(status.HTTP_409_CONFLICT, "La última operación no falló")
    return encolar(db, tenant, ultima.tipo, reintento_de=ultima.id)
```

Figura 16. plataforma/backend/plataforma/main.py, líneas 233 a 242. Reintentar vuelve a encolar la misma operación que falló.

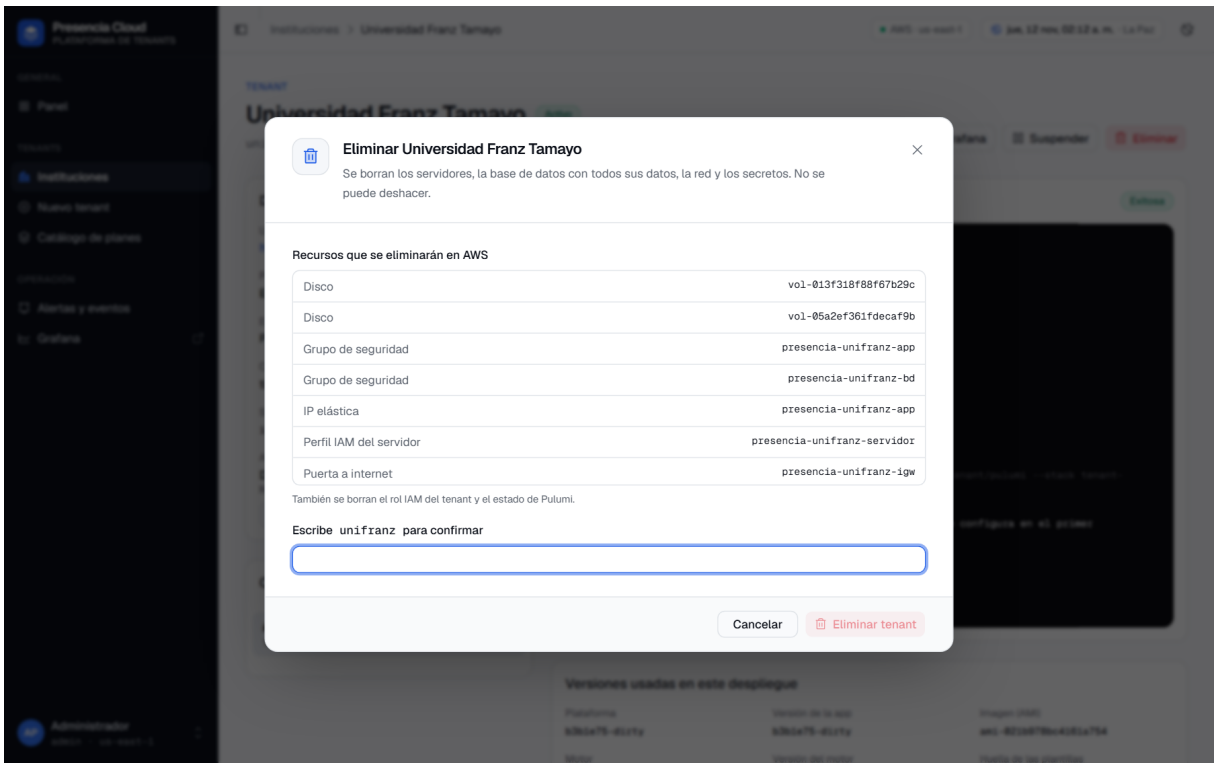


Figura 17. Antes de eliminar se listan los recursos reales y se pide el identificador.

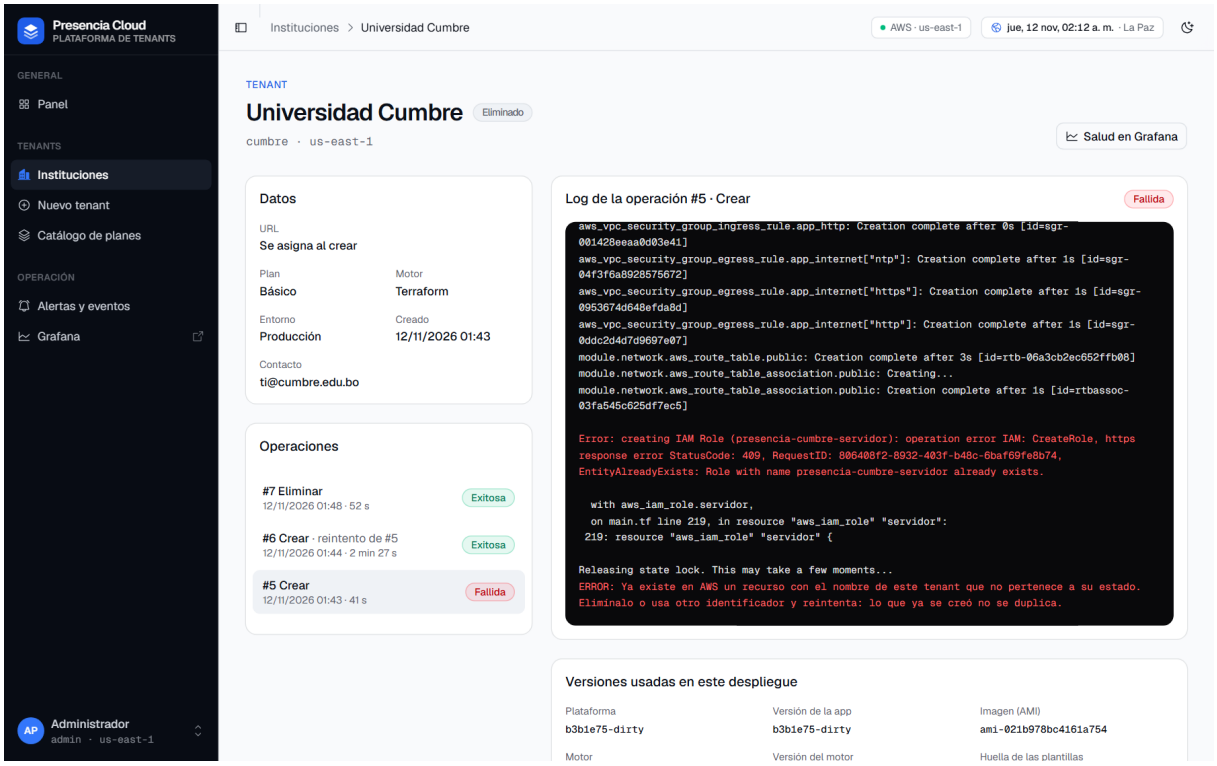


Figura 18. Operación fallida con su explicación; el reintento creó solo los 6 recursos que faltaban.

Tabla 5. Terraform y Pulumi comparados

Aspecto	Terraform	Pulumi
Lenguaje	HCL declarativo	Python
Estado	S3, tenants/<id>/	S3, pulumi/, un stack por tenant
Secretos en el estado	Protegidos por el cifrado del bucket	Cifrados además con frase de paso
Resultado	Mismos recursos, nombres y etiquetas	Mismos recursos, nombres y etiquetas

Cómo funciona por detrás

Pulumi se ejecuta por línea de comandos con un stack tenant-<id>; su configuración, con la sal de cifrado, se guarda en el volumen de datos de la plataforma. Suspender y reanudar usan boto3 sobre las instancias del tenant; la IP elástica mantiene la URL. Eliminar ejecuta destroy en el motor con que se creó el tenant y borra su estado. Reintentar repite la operación: el motor compara con su estado y solo crea lo que falta.

Monólogo

Lectura de aproximadamente 1,7 minutos (236 palabras a un ritmo de presentación de unas 140 palabras por minuto).

Yo hice el segundo motor y el ciclo de vida. El requisito pide poder elegir el motor, y nosotros ofrecemos Terraform y Pulumi. Este tenant, la Universidad Franz Tamayo, se creó con Pulumi en un plan Estándar: en el log se ve cómo Pulumi crea uno por uno los recursos.

Aquí está el programa en Python. Si lo comparan con Terraform, crea exactamente los mismos recursos, con los mismos nombres y etiquetas, y lee el mismo catálogo. La diferencia es que en Pulumi la infraestructura es código Python con funciones y condiciones normales, y cada institución es un stack cuyo estado también va a S3, con los secretos cifrados.

Ahora el ciclo de vida. Si una institución deja de usar el sistema, presiono Suspender: los servidores se apagan y dejamos de pagar el cómputo, pero los discos, los datos y la IP elástica se conservan. Al presionar Reanudar vuelve a la misma dirección y con todos sus estudiantes.

Para eliminar, la plataforma consulta en AWS todos los recursos con la etiqueta del tenant y los muestra antes de borrar; hay que escribir el identificador para confirmar. Recién ahí ejecuta destroy con el motor que corresponda.

Y si una operación falla, se puede reintentar. En esta prueba provocamos un error a mitad de la creación: al corregirlo y reintentar, el motor partió de su estado y creó solo los seis recursos que faltaban, sin duplicar la red.

5. INTEGRANTE 4 · PACKER, CLOUD-INIT Y ANSIBLE

Cubre: desplegar la aplicación y configurar su conexión con la base de datos (RF-10), y las tres herramientas de la sección 4: Packer para imágenes preconfiguradas, cloud-init para la configuración inicial y Ansible para la configuración posterior.

Qué mostrar en pantalla

1. Mostrar presencia.pkr.hcl e imagen.yml: cómo se construye la imagen dorada.
2. Mostrar la plantilla de cloud-init y el script de arranque.
3. En el log de unifranz, bajar hasta «Configuración posterior con Ansible» y el PLAY RECAP con failed=0.
4. Mostrar tenant.yml: firewall, SSH, usuario de operación y verificación.
5. Abrir la URL del tenant upsa: el login con la marca de la institución.
6. Entrar como su administrador (registro 100200300, admin1234!) y mostrar el estado de los servicios.

Tabla 6. Qué hace cada herramienta y cuándo

Momento	Herramienta	Qué hace
Una vez, antes de los tenants	Packer + Ansible	Construye la AMI con Presencia instalada y compilada (imagen.yml).
Primer arranque del servidor	cloud-init	Deja los datos del tenant y ejecuta arranque.sh: secretos, base de datos y servicios.
Después de crear el tenant	Ansible	Configura cada servidor desde la plataforma (tenant.yml).

```
build {
  sources = ["source.amazon-ebs.presencia"]

  provisioner "ansible" {
    playbook_file = "${path.root}/../ansible/imagen.yml"
    user          = "ubuntu"
    use_proxy     = false

    ansible_env_vars = [
      "ANSIBLE_HOST_KEY_CHECKING=False",
      "ANSIBLE_PIPELINING=True",
      "ANSIBLE_PYTHON_INTERPRETER=auto_silent",
      "ANSIBLE_CALLBACK_RESULT_FORMAT=yaml",
    ]
  }

  post-processor "manifest" {
    output     = "${path.root}/manifest.json"
    strip_path = true
  }
}
```

Figura 19. infraestructure/packer/presencia.pkr.hcl, líneas 84 a 104. Packer parte de Ubuntu 24.04 y provisiona con Ansible.

```
    - name: Compilar Next.js
      become_user: ubuntu
      ansible.builtin.shell: >-
        pnpm install --frozen-lockfile && pnpm build
        && cp -r public .next/standalone/
        && mkdir -p .next/standalone/.next
        && cp -r .next/static .next/standalone/.next/
      args:
        chdir: "{{ app_dir }}/frontend"
      environment: "{{ build_env }}"
      async: 1800
      poll: 15
      changed_when: true
```

Figura 20. infraestructure/ansible/imagen.yml, líneas 149 a 161. La compilación del frontend se hace una sola vez, en la imagen.

```
#cloud-config
write_files:
  - path: /etc/presencia/tenant.env
    permissions: "0644"
    content: |
      TENANT='${tenant}'
      INSTITUCION='${institucion}'
      ROL='${rol}'
      BD_MODO='${bd_modo}'
      BD_HOST='${bd_host}'
      REGION='${region}'
      VPC_CIDR='${vpc_cidr}'
      ADMIN_REGISTRO='${admin_registro}'
      ADMIN_NOMBRE='${admin_nombre}'
      ADMIN_CORREO='${admin_correo}'
      DATOS_DEMO='${datos_demo}'

runcmd:
  - [/usr/local/sbin/presencia-arranque]
```

Figura 21. infraestructure/tenant/cloud-init/servidor.yaml.tftpl, líneas 1 a 19. cloud-init solo recibe datos no secretos y llama al script.

```
  install -m 600 -o ubuntu -g ubuntu /dev/null "$APP_DIR/backend/.env"
  cat > "$APP_DIR/backend/.env" <<ENV
ENVIRONMENT=production
APP_NAME=Presencia ${INSTITUCION}
DATABASE_URL=postgresql+psycopg://${BD_USUARIO}:${BD_PASSWORD}@${bd_host}:5432/${BD_NOMBRE}?connect_timeout=3
JWT_SECRET=$(secreto jwt_secret)
DEFAULT_PASSWORD=$(secreto admin_password)
ADMIN_REGISTRATION=${ADMIN_REGISTRO}
ADMIN_EMAIL=${ADMIN_CORREO}
ADMIN_NAME=${ADMIN_NOMBRE}
TIMEZONE=America/La_Paz
ENV

  local demo=""
  [[ $DATOS_DEMO == true ]] && demo="--demo"
  sudo -u ubuntu -H bash -c "cd '$APP_DIR/backend' && .venv/bin/alembic upgrade head && .venv/bin/python -m app.seed $demo"

  sed -i '/^INSTITUCION/d' /etc/presencia-frontend.env
  {
    echo "INSTITUCION=${INSTITUCION}"
    echo "INSTITUCION_SIGLA=${TENANT^^}"
  } >> /etc/presencia-frontend.env

  systemctl enable --now presencia-backend presencia-frontend
```

Figura 22. infraestructure/tenant/scripts/arranque.sh, líneas 72 a 95. El script arma la conexión a la base con secretos leídos de SSM y aplica migraciones.

```
62      - name: Firewall - SSH solo desde la plataforma
63        community.general.ufw:
64          rule: allow
65          port: "22"
66          proto: tcp
67          src: "{{ plataforma_cidr }}"
68
69      - name: Firewall - métricas solo para Prometheus de la plataforma
70        community.general.ufw:
71          rule: allow
72          port: "{{ '9100:9101' if rol == 'app' else '9100' }}"
73          proto: tcp
74          src: "{{ plataforma_cidr }}"
75
76      - name: Firewall - web pública
77        community.general.ufw:
78          rule: allow
79          port: "80"
80          proto: tcp
81        when: rol == 'app'
82
83      - name: Firewall - PostgreSQL solo dentro de la red del tenant
84        community.general.ufw:
85          rule: allow
86          port: "5432"
87          proto: tcp
88          src: "{{ vpc_cidr }}"
89        when: rol == 'bd'
90
91      - name: Firewall - denegar todo lo demás y activar
92        community.general.ufw:
93          state: enabled
94          policy: deny
95          direction: incoming
```

Figura 23. infraestructure/ansible/tenant.yml, líneas 62 a 95. Ansible deja el firewall con lo mínimo: SSH y métricas solo para la plataforma.

```
== Configuración posterior con Ansible ==
$ ansible-playbook -i /tmp/tmp8a19wu96/hosts.ini /app/repo/infraestructure/ansible/tenant.yml -u ubuntu --private-key
PLAY [Configuración posterior de los servidores del tenant] ********************
TASK [Gathering Facts] *********************************************************
TASK [Esperar a que cloud-init termine el arranque] ****************************
TASK [Nombre del servidor] *****************************************************
changed: [unifranz-app]
changed: [unifranz-bd]
TASK [Zona horaria de Bolivia] *************************************************
changed: [unifranz-app]
changed: [unifranz-bd]
TASK [Usuario de operación sin contraseña] *************************************
changed: [unifranz-app]
changed: [unifranz-bd]
TASK [Acceso del usuario de operación con la clave de la plataforma] ***********
changed: [unifranz-app]
changed: [unifranz-bd]
TASK [Permisos del usuario de operación (solo revisar y reiniciar los servicios)] ***
changed: [unifranz-app]
changed: [unifranz-bd]
TASK [Endurecer SSH] ***********************************************************
changed: [unifranz-app]
…
changed: [unifranz-bd]
PLAY RECAP *********************************************************************
unifranz-app               : ok=16   changed=11   unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
unifranz-bd                : ok=16   changed=11   unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
Listo. Administrador del tenant: registro 200300400
```

Figura 24. Salida real de Ansible sobre los dos servidores del tenant unifranz.

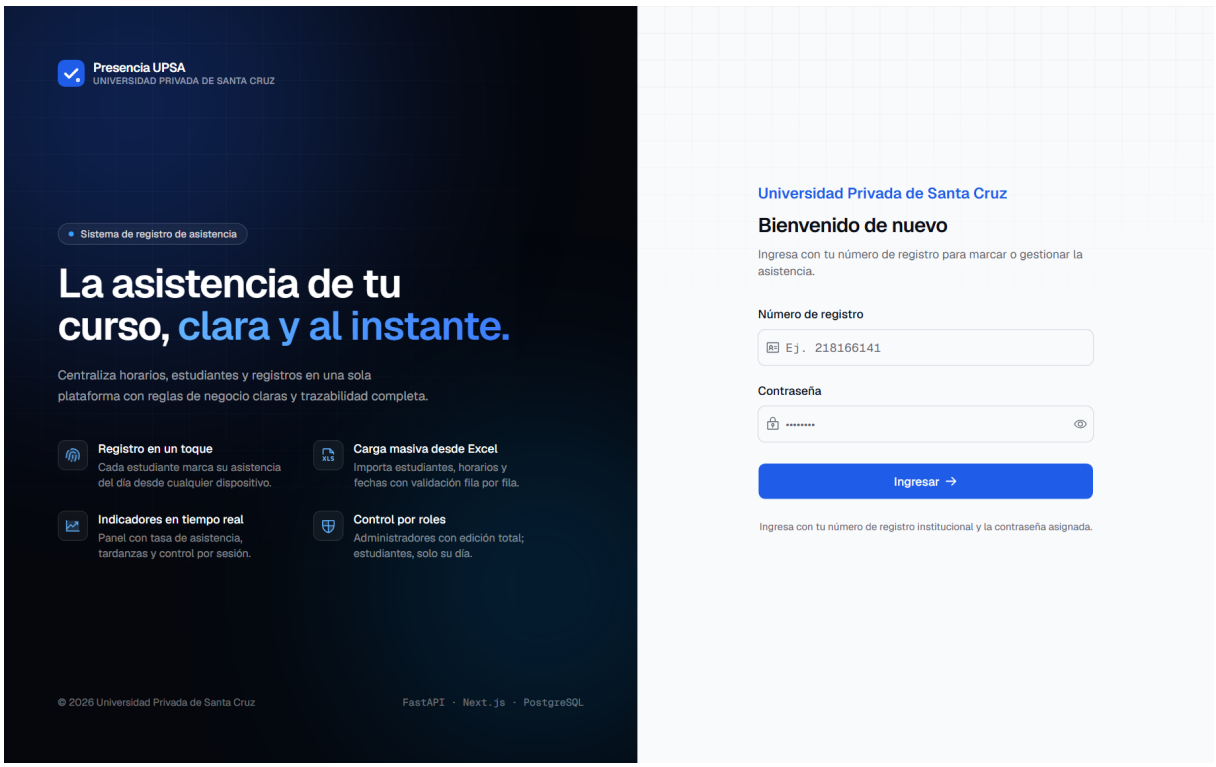


Figura 25. El tenant se presenta con el nombre de su institución.

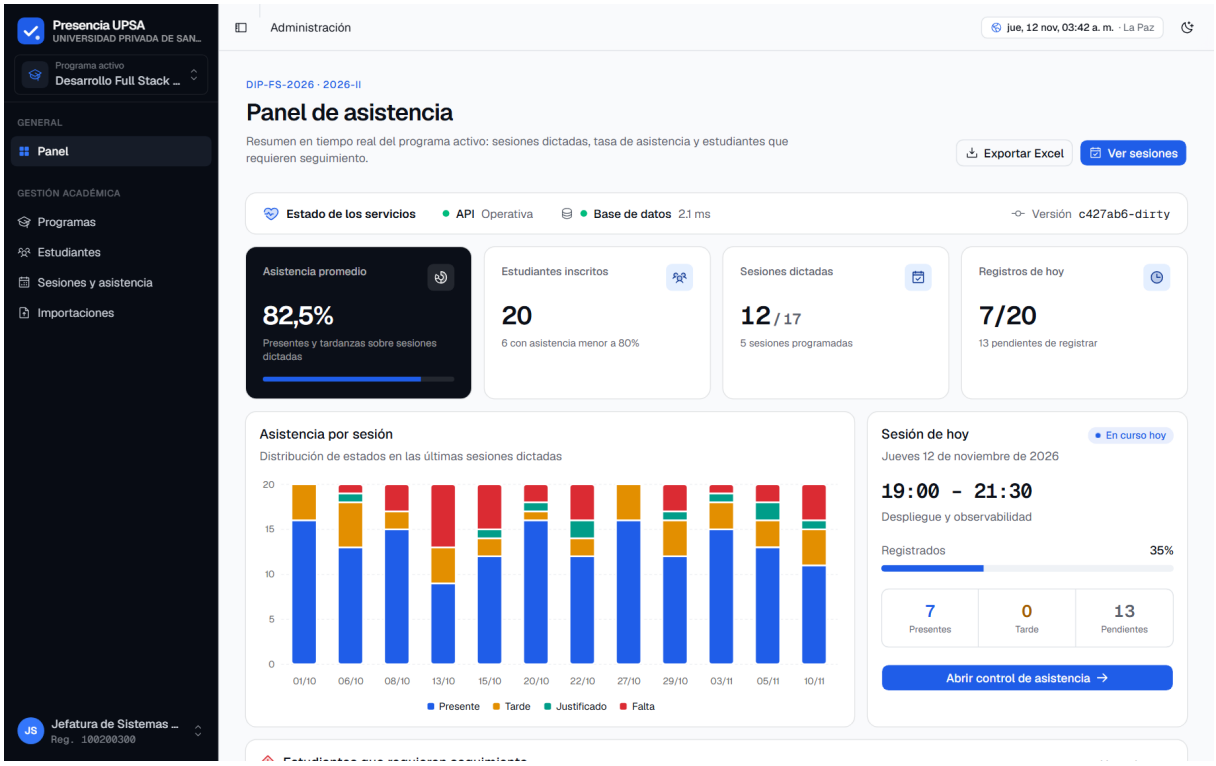


Figura 26. El administrador del tenant ve el estado de sus servicios.

Cómo funciona por detrás

La AMI se construye desde WSL con packer build. Terraform o Pulumi lanzan los servidores desde esa imagen con la plantilla de cloud-init, que escribe /etc/presencia/tenant.env y ejecuta el script de arranque. El script obtiene las contraseñas con el rol IAM del servidor, configura PostgreSQL según el plan, escribe la marca de la institución y enciende los servicios. Cuando la app responde, el worker arma el inventario con las IP de salida y ejecuta ansible-playbook por SSH con la llave de la plataforma.

Tabla 7. Comandos

Para qué	Comando
Construir la imagen dorada (WSL)	`cd infraestructure/packer && packer build -var version_app=$(git describe --always) .`
Ver el arranque en un servidor	`sudo tail -f /var/log/cloud-init-output.log`
Probar el playbook a mano	`ansible-playbook --syntax-check -i localhost, tenant.yml`

Monólogo

Lectura de aproximadamente 1,7 minutos (233 palabras a un ritmo de presentación de unas 140 palabras por minuto).

Yo trabajé con Packer, cloud-init y Ansible, que son las tres herramientas que preparan cada servidor, y cada una tiene un momento distinto.

Primero Packer. En vez de instalar todo cada vez que se crea un tenant, Packer levanta una máquina temporal y la configura con un playbook de Ansible: instala PostgreSQL, Nginx y Node, copia Presencia y compila el frontend. El resultado es una imagen dorada. Gracias a eso un tenant está listo en unos cinco minutos.

Segundo, cloud-init, la configuración inicial. Cuando el servidor arranca por primera vez, cloud-init deja los datos del tenant y ejecuta nuestro script de arranque. Ese script lee las contraseñas desde SSM Parameter Store con el rol del servidor, así ningún secreto viaja en la plantilla. Luego configura la base según el plan, aplica las migraciones, crea el administrador de la institución y enciende la aplicación con la marca de esa institución, como se ve en este login.

Tercero, Ansible para la configuración posterior, que es lo que pide el requisito. Cuando la app ya responde, la plataforma ejecuta este playbook por SSH, con su propia llave y el puerto 22 abierto solo para su IP. Configura el firewall, endurece SSH, crea un usuario de operación con permisos mínimos, activa las actualizaciones de seguridad y verifica que los servicios estén corriendo. En el log se ve el resultado: failed igual a cero en los dos servidores.

6. INTEGRANTE 5 · GRAFANA

Cubre: integrar Grafana (RF-25), disponibilidad, CPU, memoria, latencia, errores del backend y estado de la base de datos (RF-26), métricas por tenant y por entorno (RF-27), alertas (RF-28) y alertas y eventos en la plataforma (RF-29).

Qué mostrar en pantalla

1. Abrir `http://3.235.12.162/grafana/d/salud-tenants` y explicar los filtros Entorno y Tenant.
2. Recorrer los paneles: estado de cada servidor, estado de la base de datos, CPU, memoria, disco, peticiones, errores 5xx y latencia.
3. Provocar una alerta: en Session Manager del servidor de un tenant ejecutar `sudo systemctl stop presencia-backend` y pausar la grabación unos 3 minutos.
4. Mostrar la alerta disparada en Grafana y en Alertas y eventos de la plataforma, con el tenant y la hora.
5. Restaurar con `sudo systemctl start presencia-backend`.

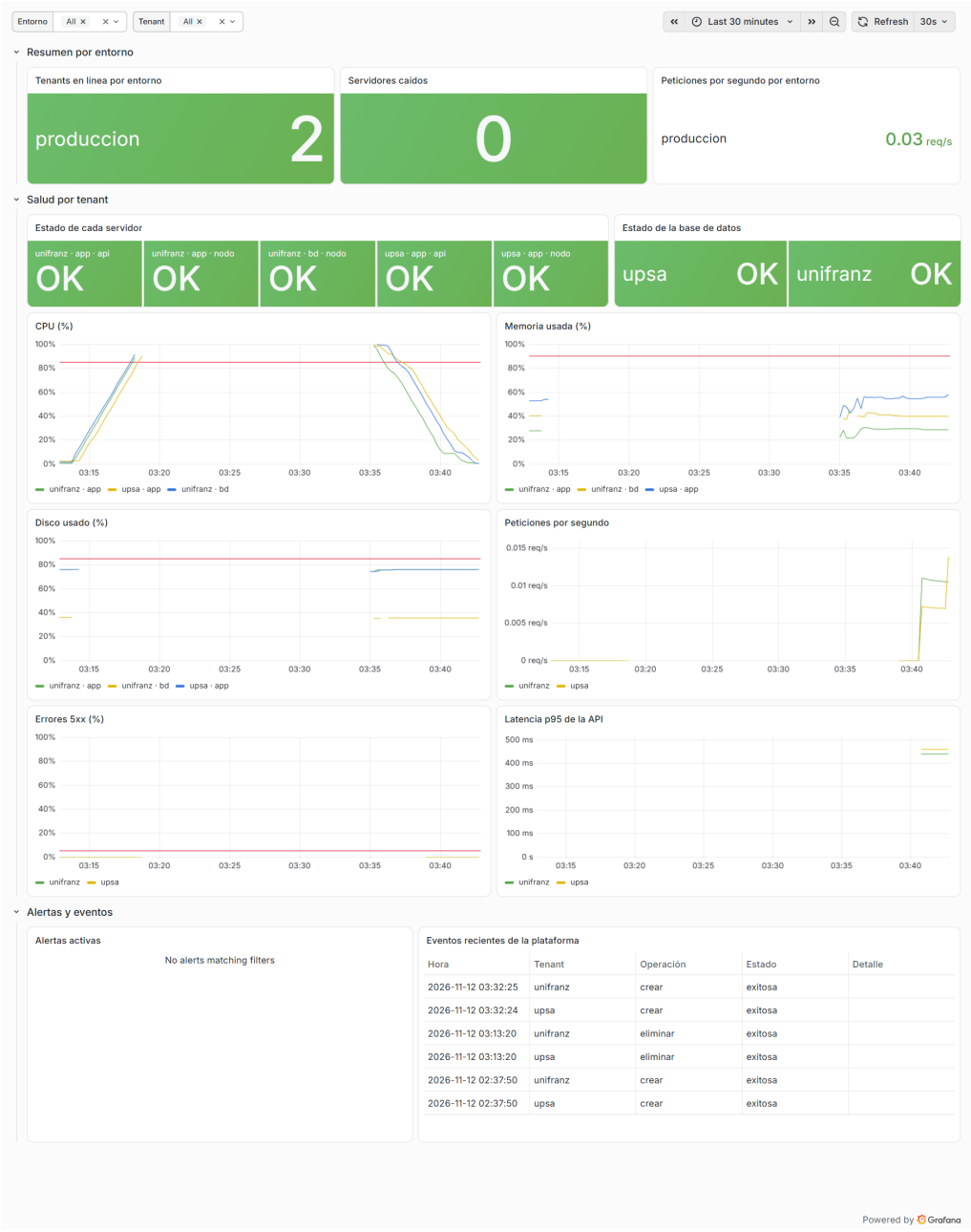


Figura 27. Tablero «Salud de los tenants».

```
  - job_name: nodo
    ec2_sd_configs: &ec2
      - region: us-east-1
        refresh_interval: 30s
        filters:
          - name: tag:Proyecto
            values: [presencia-plataforma]
          - name: instance-state-name
            values: [running]
    relabel_configs:
      - source_labels: [__meta_ec2_public_ip]
        regex: (.+)
        target_label: __address__
        replacement: ${1}:9100
      - source_labels: [__meta_ec2_tag_Tenant]
        target_label: tenant
      - source_labels: [__meta_ec2_tag_Plan]
        target_label: plan
      - source_labels: [__meta_ec2_tag_Entorno]
        target_label: entorno
      - source_labels: [__meta_ec2_tag_Role]
        target_label: rol
      - source_labels: [__meta_ec2_tag_Name]
        target_label: instance

  - job_name: api
```

Figura 28. plataforma/monitoreo/prometheus.yml, líneas 6 a 31. Prometheus descubre los servidores de AWS por etiqueta y copia sus etiquetas.

```
BASE_DATOS_ARRIBA = Gauge(
    "presencia_base_datos_arriba", "1 si la API llega a la base de datos del tenant, 0 si no"
)
BASE_DATOS_ARRIBA.set_function(lambda: 1.0 if estado_base_datos()[0] else 0.0)
```

Figura 29. backend/app/main.py, líneas 26 a 29. Presencia expone sus métricas y si llega a su base de datos.

```
- uid: tenant-caido
  title: Tenant caído
  condition: C
  for: 2m
  noDataState: OK
  execErrState: Error
  labels:
    severidad: critica
  annotations:
    summary: El servidor {{ .Labels.rol }} del tenant {{ .Labels.tenant }} no responde
      ({{ .Labels.job }})
  data:
  - refId: A
    relativeTimeRange:
      from: 600
      to: 0
    datasourceUid: prometheus
    model:
      refId: A
      expr: min by (tenant, rol, job) (up)
      instant: true
      range: false
  - refId: C
    datasourceUid: __expr__
    model:
      refId: C
      type: threshold
      expression: A
      conditions:
      - evaluator:
```

Figura 30. plataforma/monitoreo/grafana/alerting/reglas.yml, líneas 8 a 37. Una regla de alerta provisionada como código.

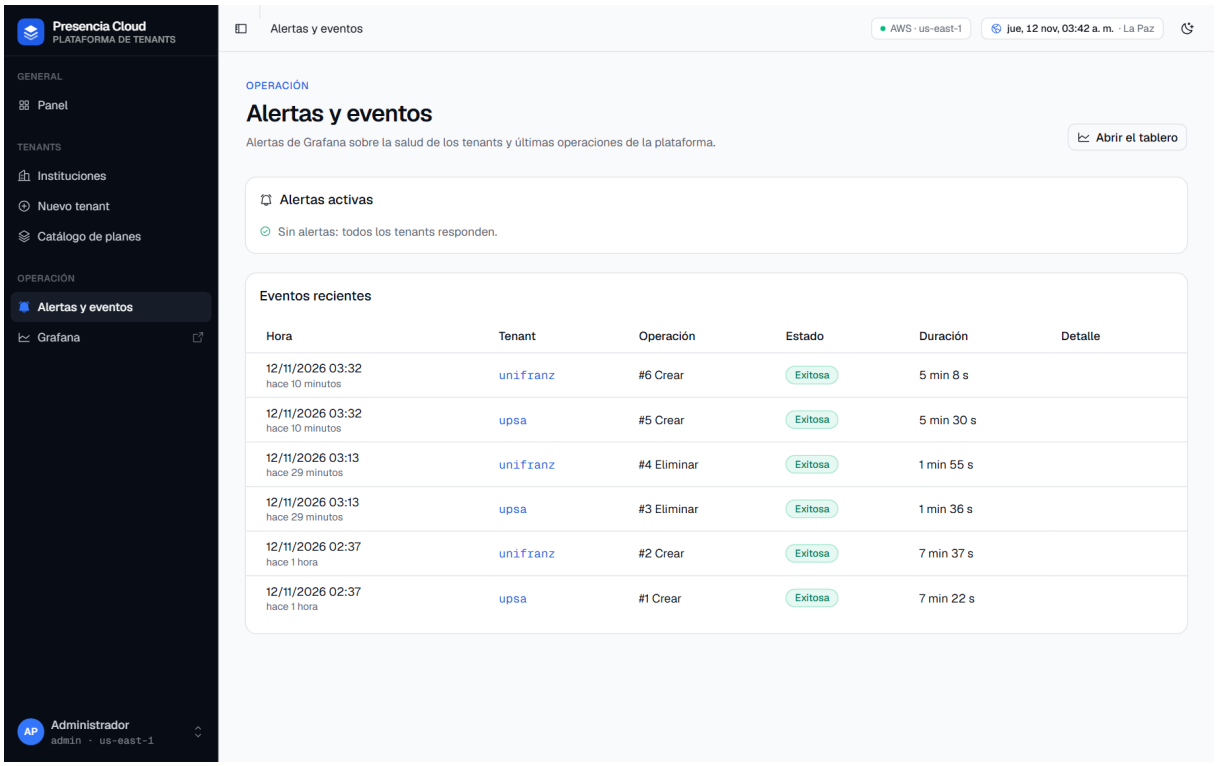


Figura 31. Alertas y eventos recientes en la plataforma.

Tabla 8. Alertas configuradas

Alerta	Condición
Tenant caído	Un servidor no responde durante 2 minutos.
Base de datos caída	La API del tenant no llega a su base durante 2 minutos.
CPU alta	Más de 85 % durante 5 minutos.
Memoria alta	Más de 90 % durante 5 minutos.
Disco casi lleno	Más de 85 % durante 5 minutos.
Errores 5xx	Más del 5 % de las peticiones durante 5 minutos.

Cómo funciona por detrás

Cada servidor trae node_exporter en la imagen (CPU, memoria y disco) y la API de Presencia expone /metrics con peticiones, códigos de respuesta, latencia y el estado de la base. Nginx publica esas métricas en el puerto 9101, abierto solo para la plataforma. Prometheus usa el rol IAM de la plataforma para listar las instancias encendidas con la etiqueta Proyecto=presencia-plataforma, así un tenant nuevo aparece solo y uno suspendido desaparece sin generar falsas alarmas. Grafana se configura completo desde archivos y se sirve bajo /grafana.

Monólogo

Lectura de aproximadamente 1,7 minutos (239 palabras a un ritmo de presentación de unas 140 palabras por minuto).

Yo integré el monitoreo con Prometheus y Grafana. Aprovisionar no alcanza: hay que saber si cada institución está funcionando bien.

Cada servidor sale de la imagen con node_exporter, que mide CPU, memoria y disco, y la API de Presencia publica sus propias métricas: cuántas peticiones recibe, cuántas fallan, cuánto tardan y si llega a su base de datos. Prometheus no necesita una lista de servidores: usa el rol de AWS de la plataforma para encontrar las instancias encendidas por su etiqueta, y convierte las etiquetas de AWS en etiquetas de las métricas. Por eso, cuando se crea un tenant, aparece solo en el tablero.

Este es el tablero de Grafana. Arriba, el resumen por entorno; abajo, cada servidor con OK o CAÍDO, el estado de la base de datos, el uso de recursos, las peticiones, los errores del backend y la latencia. Con estos filtros veo un tenant o un entorno en particular.

Las alertas también están definidas como código. Para probarlas, detengo el backend de un tenant desde Session Manager. A los dos minutos se dispara la alerta Tenant caído, con el nombre del tenant y la hora, y la misma alerta aparece en la plataforma, en Alertas y eventos, junto al historial de operaciones. Vuelvo a encender el servicio y la alerta se resuelve sola.

Grafana se publica en la misma dirección de la plataforma, bajo la ruta grafana, para que se pueda consultar sin instalar nada.

7. INTEGRANTE 6 · SEGURIDAD Y DOCUMENTACIÓN

Cubre: los requisitos no funcionales (aislamiento, secretos fuera del código, operaciones repetibles, estado protegido, errores sin secretos), la verificación de aislamiento (CU-14) y los entregables de documentación.

Qué mostrar en pantalla

1. En el log de un tenant, mostrar el escaneo con gitleaks y checkov antes de aprovisionar.
2. Mostrar en GitHub el workflow «Secretos · gitleaks» en verde.
3. En la consola de AWS, mostrar los parámetros /presencia/tenants/ en SSM Parameter Store, cifrados.
4. En Session Manager de la plataforma, ejecutar la prueba de aislamiento y mostrar «20 de 20».
5. Mostrar el documento técnico, los diagramas y este guion.

```
Tenant upsa (Universidad Privada de Santa Cruz) · plan basico · motor terraform
Imagen: ami-0ff84e07d88764d93 (presencia-20261112-0713, app c427ab6)
Métricas abiertas solo para la IP de la plataforma: 3.235.12.162/32
Versiones: {'plataforma': 'c427ab6', 'app': 'c427ab6', 'ami': 'ami-0ff84e07d88764d93', 'motor': 'terraform', 'version_motor': '1.16.6', 'ansible': 'ansible [core 2.20.1]', 'plantillas': '33febf15f76c'}
== Escaneo de secretos (gitleaks) ==
$ gitleaks dir /app/repo/infraestructure/tenant --no-banner --no-color --redact --verbose
7:32AM INF scanned ~140167 bytes (140.17 KB) in 166ms
7:32AM INF no leaks found
$ gitleaks dir /tmp/tmpg928ek2i/tenant.json --no-banner --no-color --redact --verbose
7:32AM INF scanned ~532 bytes (532 bytes) in 20.2ms
7:32AM INF no leaks found
Sin secretos en las plantillas ni en la configuración del tenant.
== Escaneo de configuración (checkov) ==
$ checkov -d /app/repo/infraestructure/tenant/terraform --framework terraform --compact --quiet --soft-fail --skip-download
terraform scan results:

Passed checks: 107, Failed checks: 22, Skipped checks: 0

Advertencia: checkov marcó 22 recomendaciones (no bloquean).
```

Figura 32. Salida real del escaneo de seguridad antes de crear el tenant upsa.

```python
def escanear(variables: dict, log: Log) -> None:
    carpeta = get_settings().tenant_dir
    with tempfile.TemporaryDirectory() as tmp:
        config = Path(tmp) / "tenant.json"
        config.write_text(json.dumps(variables, indent=2), encoding="utf-8")

        log("== Escaneo de secretos (gitleaks) ==")
        for objetivo in (carpeta, config):
            try:
                ejecutar(
                    ["gitleaks", "dir", str(objetivo), "--no-banner", "--no-color", "--redact", "--verbose"],
                    log,
                )
            except ErrorOperacion as error:
                raise ErrorOperacion(
                    f"Se encontraron credenciales o secretos escritos en {objetivo}. Bórralos y "
                    "guárdalos en SSM Parameter Store antes de aprovisionar."
                ) from error
        log("Sin secretos en las plantillas ni en la configuración del tenant.")
```

Figura 33. plataforma/backend/plataforma/escaneo.py, líneas 13 a 31. Si gitleaks encuentra un secreto, el aprovisionamiento se detiene con un mensaje claro.

```
211 resource "aws_ssm_parameter" "secretos" {
212   for_each = {
213     bd_password    = random_password.bd.result
214     jwt_secret     = random_password.jwt.result
215     admin_password = var.contrasena_inicial
216   }
217
218   name  = "/presencia/tenants/${var.tenant}/${each.key}"
219   type  = "SecureString"
220   value = each.value
221 }
```

Figura 34. infraestructure/tenant/terraform/main.tf, líneas 211 a 221. Secretos generados por el motor, cifrados en SSM, y un rol que solo lee los de su propio tenant.

```
1  name: Secretos · gitleaks
2
3  on:
4    pull_request:
5    push:
6      branches: [main]
7    workflow_dispatch:
8
9  permissions:
10   contents: read
11
12 jobs:
13   gitleaks:
14     name: gitleaks (historial completo)
15     runs-on: ubuntu-24.04
16     timeout-minutes: 5
17     env:
18       GITLEAKS_VERSION: 8.30.1
19     steps:
20       - uses: actions/checkout@v7
21         with:
22           fetch-depth: 0
23
24       - name: Instalar gitleaks
25         run: |
26           curl -fsSL "https://github.com/gitleaks/gitleaks/releases/download/v${GITLEAKS_VERSION}/gitleaks_${GITLEAKS_VERSION}_linux_x64.tar.gz" \
27             | tar -xz -C /usr/local/bin gitleaks
28
29       - name: Buscar secretos
30         run: gitleaks git . --no-banner --redact --verbose
```

Figura 35. .github/workflows/secretos.yml, líneas 1 a 30. gitleaks revisa todo el historial en cada push y pull request.

```
$ docker compose exec backend python scripts/probar_aislamiento.py upsa unifranz

1. Cada administrador entra solo a su propio tenant (http://3.218.155.205 y http://34.198.102.24)
  OK El admin de upsa entra a upsa
  OK El admin de unifranz entra a unifranz
  OK Las credenciales de upsa no sirven en unifranz

2. El token de un tenant no vale en el otro (cada uno firma con su propio JWT_SECRET)
  OK upsa lee sus estudiantes con su token  (26 estudiantes)
  OK El token de upsa es rechazado por unifranz  (HTTP 401)
  OK Cada base de datos tiene sus propios datos  (upsa: 26 estudiantes, unifranz: 26 estudiantes)

3. Desde afuera no se llega a la base de datos ni a la API interna
  OK upsa 3.218.155.205:5432 cerrado
  OK upsa 3.218.155.205:8000 cerrado
  OK unifranz 34.198.102.24:5432 cerrado
  OK unifranz 34.198.102.24:8000 cerrado
  OK unifranz 54.175.57.252:5432 cerrado
  OK unifranz 54.175.57.252:8000 cerrado

4. Desde el servidor de upsa no se llega a los servidores de unifranz (tampoco por SSH)
  OK upsa → unifranz 34.198.102.24:5432 cerrado
  OK upsa → unifranz 34.198.102.24:8000 cerrado
  OK upsa → unifranz 34.198.102.24:9100 cerrado
  OK upsa → unifranz 34.198.102.24:22 cerrado
  OK upsa → unifranz 54.175.57.252:5432 cerrado
  OK upsa → unifranz 54.175.57.252:8000 cerrado
  OK upsa → unifranz 54.175.57.252:9100 cerrado
  OK upsa → unifranz 54.175.57.252:22 cerrado

20 de 20 comprobaciones correctas
```

Figura 36. Salida real de la prueba de aislamiento entre upsa y unifranz.

Tabla 9. Controles de seguridad

Riesgo	Control
Un tenant accede a otro	VPC propia, grupos de seguridad, firewall ufw y clave JWT distinta por tenant.
Secretos en el repositorio	Generados por el motor, cifrados en SSM; gitleaks antes de cada despliegue y en CI.
Credenciales de AWS expuestas	La plataforma usa un rol IAM, sin claves de acceso.
Estado del motor comprometido	Bucket cifrado, versionado, sin acceso público y con bloqueo.
Errores que filtran datos	Los motores ocultan valores sensibles y gitleaks enmascara lo que detecta.

Tabla 10. Comandos

Para qué	Comando
Prueba de aislamiento (en la plataforma)	`cd /opt/presencia/plataforma && sudo docker compose exec -e PLATAFORMA_URL=http://localhost:8000 backend python scripts/probar_aislamiento.py upsa unifranz`
Escanear el historial del repositorio	`gitleaks git . --redact`
Regenerar la documentación	`uv run --project backend --with pyyaml python docs/build_pdf_plataforma.py`

Cómo funciona por detrás

Antes de cada aprovisionamiento, el worker escanea las plantillas y la configuración del tenant con gitleaks y revisa el Terraform con checkov. Los secretos nunca se escriben: el motor los genera, los guarda cifrados en SSM y cada servidor los lee con su rol. La prueba de aislamiento entra a cada tenant con sus credenciales, intenta usar el token de uno en el otro, prueba los puertos desde afuera y, por Session Manager, desde el servidor de un tenant hacia los del otro.

Monólogo

Lectura de aproximadamente 1,8 minutos (251 palabras a un ritmo de presentación de unas 140 palabras por minuto).

Yo me encargué de la seguridad y de la documentación. Lo primero es que ningún secreto esté en el código. Antes de aprovisionar, la plataforma escanea las plantillas y la configuración con gitleaks; si encuentra una contraseña escrita, se detiene y lo explica. También revisa el Terraform con checkov, que marca recomendaciones de buenas prácticas. Y en GitHub, cada push vuelve a revisar todo el historial.

Las contraseñas de cada tenant las genera el propio motor y se guardan cifradas en SSM Parameter Store. Cada servidor tiene un rol IAM que solo puede leer los secretos de su propio tenant. La plataforma tampoco tiene claves de AWS: usa el rol de su instancia con los permisos mínimos.

Para demostrar el aislamiento escribimos una prueba automática. Entra a cada tenant con su administrador, intenta usar las credenciales y el token de un tenant en el otro, verifica que la base de datos y la API interna no son accesibles desde afuera, y desde el servidor de un tenant intenta llegar a los puertos del otro, incluido SSH. El resultado es veinte de veinte: cada institución está aislada en su red, con su base y sus propias claves.

El estado de los motores está en un bucket cifrado, versionado y sin acceso público. Finalmente, preparamos el documento técnico con la arquitectura, el catálogo de planes, el manual de uso y las pruebas, los diagramas en PlantUML y este guion. En resumen, la plataforma crea, opera y monitorea instituciones aisladas, todo definido como código.